



← Back to student guides

 DVC

Data versioning

3 levels

113 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

CODE

POINTER

CACHE

REMOTE



Quick Start

 Open PDF

A one-page starter for this tool. Scroll through it here, or open it full screen.





Git-LFS

Solves the size problem, but still charges for bandwidth/storage per repo and has zero awareness of pipelines — it does



MLOps MENA Community

THE FIX, IN ONE LINE

DVC keeps the heavy data on remote storage like GCS, Azure — and leaves behind a tiny pointer in Git that holds only a hash.

Git for code. DVC for data. Same commands, same habits, one workflow.



MLOps MENA Community

THE PROBLEM → THE FIX

How Git and DVC Divide the Work



16

sections

19

examples

This is part three of three. It closes the series by taking **every topic from Beginner and Mid one level further** (each one now has a security, scale, or ownership dimension) and adds the work you own when data versioning is your responsibility rather than your tool.

Up to this point the questions have been about making a pipeline reproduce. From here they are different: who can read and who can overwrite the data, whether a result from eighteen months ago can still be reconstructed and proven, what your storage costs, what happens when a customer asks you to delete their records from an immutable content-addressed store, and where DVC stops being the right tool.

Where this picks up

Topic from earlier levels	What this level adds
Remotes	The trust model : who reads, who writes, and credentials that are never stored
<code>.dvc/config.local</code>	OIDC and workload identity, so there is nothing to leak
The cache is immutable	Object lock, retention, versioning, and what they do to <code>dvc gc</code>
<code>push: false</code> , remote tiering	A real cost model: storage class, egress, request charges, lifecycle
Hashing performance	Terabyte scale, sharding strategy, and when to stop using DVC
<code>dvc import</code>	Data registries as a product , with a release cadence and consumers



dvc . LOCK	Lineage, provenance, and audit trails that satisfy a regulator
Deleting data	GDPR erasure against a content-addressed store: the hard problem
CI	Self-hosted and GPU runners, fork trust, and CML reporting
Experiments	Fleet-level tracking, model registries, and promotion gates
Debugging	Incident playbooks: corrupted cache, lost remote, unreproducible model
new	Access model · retention policy · cost governance · platform ownership · where DVC ends

I am starting with the trust model, because every other decision in this track depends on it.

The trust model: the bucket decides, not DVC

A DVC remote is a storage bucket. Everything about who can do what to your data is decided in the storage provider, not in DVC, and DVC's design means the consequences differ from what people expect.

PRINCIPALS, AND THE PERMISSION EACH ONE NEEDS

Developer

Read everywhere · write on a `dev` prefix only

CI on a protected branch

Read + write for `dvc push` . Never delete

CI on a fork PR

Read only. Any write is an open door



`ml-data-prod/dvc/`

read: everyone · write: CI on main · **delete: nobody**

`ml-data-dev/dvc/`

read+write: developers · delete: lifecycle only

`ml-data-public/dvc/`

read: anonymous and fork CI · write: CI on main



Write ⇒ silent overwrite

An object at a hash path can be replaced. Every commit referencing it now resolves to wrong bytes, with no error

`verify true`

Re-hash on download, so corruption is refused instead of restored

`gc --cloud`

Permanent deletion, scoped by whatever the running client can see

Why write matters more than it looks: content addressing makes an identical push a no-op, which reads as harmless, but the same permission lets a client put different bytes at that hash. Bucket versioning is what makes that survivable, and it is the highest-priority change on this page.

Actor	Needs	Should not have
A developer on the team	Read on the remote, write on a dev prefix	Write on the production data prefix
CI on a protected branch	Read, and write for <code>dvc push</code>	Delete
CI on a fork pull request	Read only	Any write at all
A training runner	Read, and write for artifacts	Delete



A downstream consumer	Read on specific prefixes	Anything else
The retention job	Delete, scoped and audited	Write

Three properties of DVC's model make the access question sharper than it looks:

Content-addressed writes are silent overwrites. Pushing an object writes to a path derived from its hash. Identical content is a no-op, but a client that can write can also **replace** an object at a hash path with different bytes, and every commit referencing that hash now resolves to the wrong data with no error anywhere. That is the reason write access matters more than it first appears.

A pointer is not proof. `dvc.lock` records the hash DVC expected. Nothing verifies the bytes on download unless you ask it to:

BASH

Copy

```
dvc remote modify origin verify true      # re-hash after every download
dvc pull --verify                          # or per-invocation
```

Deletion is the dangerous permission. `dvc gc --cloud` deletes objects on the remote, permanently, based on what the running client considers reachable. A developer with delete permission and a stale clone can remove data every other branch depends on.

The single most dangerous command in DVC

`dvc gc --cloud --workspace` deletes every remote object not needed by the **currently checked-out** revision. On a repository with twenty branches that is a catastrophic, unrecoverable data loss, and it exits zero. Remove delete permission from human credentials entirely, do retention with a separate audited job, and enable object versioning so a mistake is survivable.

The access shape that works:



```
s3://ml-data-prod/dvc/      ← read: everyone    write: CI on main only  delete: no
s3://ml-data-dev/dvc/      ← read+write: developers delete: l
s3://ml-data-public/dvc/   ← read: anonymous / fork CI write: CI
```

► TRY IT

1. List who currently holds write access to your data remote. Include CI, service accounts, and anyone with an admin role.
2. Check whether any human credential can delete objects. If so, write down what would happen if `dvc gc --cloud` ran with it.
3. Turn on `verify true` on a remote, corrupt one object in a test bucket, and watch `dvc pull` refuse it.
4. Confirm bucket versioning is enabled. If it is not, that is your highest-priority change.

Expected result – usually a longer write list than expected, and at least one credential that can delete. Step three is the demonstration that matters: without `verify`, a corrupted object is restored silently and reported as success.

Credentials that are never stored

Mid put credentials in `.dvc/config.local`. At this level the goal is that there is nothing to store.

```
.github/workflows/pipeline.yml
```

Copy

```
permissions:
  id-token: write          # request an OIDC token
  contents: read

steps:
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${vars.DVC_ROLE_ARN} # trust policy pins repo + branch
```



The role's trust policy is where the security lives, and it should pin the subject as narrowly as your workflow allows:

IAM trust policy condition

Copy

```
{
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:sub":
        "repo:my-org/my-repo:ref:refs/heads/main"
    }
  }
}
```

Mechanism	Stored secret	Scope
OIDC / workload identity	None	A specific repo, branch, or environment
Instance / pod identity	None	A specific machine or workload
Short-lived STS token	None persisted	Minutes
Long-lived access key	Yes	Whatever the key allows, until rotated

For humans, the equivalent is the provider's own SSO: `aws sso login` , `gcloud auth application-default login` , an SSH agent for `ssh://` remotes. DVC picks these up automatically because it uses each provider's standard credential chain.

BASH

Copy

```
dvc remote modify --local origin profile ml-readonly    # a named SSO profile
dvc remote modify --local origin ssh_private_key_path ~/.ssh/id_dvc
dvc doctor                                              # confirm the driver sees creden
```

A fork pull request must never hold a write credential



read-only remote or no remote at all, and split the workflow so the privileged part (`dvc push` , promotion, registry updates) never executes fork-supplied code.

► TRY IT

1. Replace one stored access key in CI with OIDC and confirm `dvc pull` still works.
2. Read the role's trust policy and check whether the `sub` condition pins a branch or accepts any ref.
3. Try to assume that role from a throwaway branch and confirm it is refused.
4. Delete the old secret from your CI settings and confirm nothing breaks.

Expected result – a pipeline with no stored credential, and a refusal when the branch does not match. Step three is the one that proves the trust policy is scoped rather than present.

Immutability, retention, and object lock

DVC's cache is append-only by convention. Making it append-only by **enforcement** is a storage-side decision, and it is what turns "we can reproduce that" into something you can defend.

BASH

Copy

```
# S3: versioning plus governance-mode object lock
aws s3api put-bucket-versioning --bucket ml-data-prod \
  --versioning-configuration Status=Enabled

aws s3api put-object-lock-configuration --bucket ml-data-prod \
  --object-lock-configuration '{
  "ObjectLockEnabled":"Enabled",
  "Rule":{"DefaultRetention":{"Mode":"GOVERNANCE","Days":365}}
}'
```



Bucket versioning	An overwrite or delete is recoverable	Storage for old versions
Object lock, governance mode	Deletion needs a specific privilege	Operational friction
Object lock, compliance mode	Deletion is impossible until expiry, for anyone	Cannot be undone: including by you
Lifecycle to cold storage	Cheaper long-term retention	Retrieval latency and cost
MFA delete	A second factor for destructive operations	Slower emergency response

The interaction with `dvc gc` is the part people miss:

✓ **Retention by lifecycle policy**

- Runs in the storage provider, audited
- Rules are declarative and reviewable
- Cannot be triggered by a stale clone
- Survivable via versioning

✗ **Retention by `dvc gc --cloud`**

- Reachability computed by whoever ran it
- A stale or shallow clone deletes live data
- No audit trail beyond bucket logs
- Exits zero after destroying branches' data

Compliance-mode object lock and GDPR are in direct tension

Compliance mode means nobody can delete an object before expiry, including you, and including in response to a lawful erasure request. If you are subject to both retention and erasure obligations, the resolution is architectural: keep personal data out of the immutable store, or store it encrypted per-subject so erasure means destroying a key. Decide this before enabling compliance mode, because it cannot be walked back.

► **TRY IT**

1. Enable versioning on a test bucket, push data, overwrite an object by hand, and recover the previous version.
2. Enable governance-mode object lock and try to delete an object without the bypass privilege.



proposes to remove.

Expected result – a recovered overwrite and a refused deletion, the two properties that make a data remote trustworthy. Step four is the reality check: the number is usually large enough to make you want a lifecycle policy instead.

Cost: what your data costs

Mid introduced `push: false` as a cost lever. "Storage is cheap" stops being true once a team runs sweeps, so the full cost model matters.

Four charges apply, and only the first is the one people think about:

Charge	Driven by	Typical surprise
Storage	Bytes × time × class	Every experiment's model, kept forever
Requests	PUT/GET count	A million small files costs more in requests than in bytes
Egress	Bytes leaving the region or cloud	CI in one region pulling from a bucket in another
Retrieval	Restoring from a cold class	A "cheap archive" that is expensive to read

The arithmetic that changes behaviour: a 2 GB model, thirty experiments per sweep, four sweeps a month, kept for a year is roughly 2.9 TB of accumulated artifacts for one project. At standard-class pricing that is a real line item, and almost none of it will ever be read again.

Five levers, roughly in order of return:

01 `push: false` on regenerable intermediates



02 Garbage-collect experiments on a schedule

`dvc exp remove` plus a lifecycle rule. Unreferenced sweep artifacts are the biggest source of silent growth.

03 Lifecycle to a colder class

Objects untouched for 90 days move to infrequent-access or archive. Reproducibility is preserved; only retrieval latency changes.

04 Shard small files

A million 4 KB objects is a request-count problem and a hashing problem. Tar or Parquet shards fix both, and speed up your data loader as a bonus.

05 Co-locate compute and storage

Egress from another region or another cloud can exceed the storage cost outright. Put the runner where the bucket is.

BASH

Copy

```
# Where is the money? Object counts and sizes by prefix
aws s3 ls --summarize --human-readable --recursive s3://ml-data-prod/dvc/ | tail -3

# Lifecycle: standard → IA at 90 days → Glacier at 365
aws s3api put-bucket-lifecycle-configuration --bucket ml-data-prod \
  --lifecycle-configuration file://lifecycle.json

# DVC-side hygiene
dvc exp remove --all
dvc gc --dry-run --all-commits --cloud
du -sh .dvc/cache
```

Make cost visible or it will not be managed

Tag the bucket per team or project and put the monthly figure somewhere the team sees it. Storage growth is invisible until it is a budget conversation, and by then a year of



► TRY IT

1. Measure your remote: total size, object count, and size distribution by prefix.
2. Estimate what fraction is experiment artifacts nobody has read in three months.
3. Identify your largest `outs` and ask, for each, whether it is regenerable in under ten minutes. Mark those `push: false`.
4. Write a lifecycle policy and calculate the annual saving.

Expected result – usually a majority of bytes in artifacts nobody will read again, and a clear list of outputs that should never have been pushed. That fraction in step two is the number to take to a planning conversation.

Scale: terabytes and millions of files

DVC handles large data well and many small files badly. Knowing exactly where the wall is means you can design around it instead of hitting it.

Shape	DVC behaviour	What to do
100 files × 10 GB	Fine. Hashing is I/O-bound and parallel	Nothing
1M files × 4 KB	Slow <code>add</code> , huge <code>.dir</code> , request-heavy transfers	Shard into tars, Parquet, WebDataset, LMDB
One 5 TB file	Fine to track, painful to move	Split into shards so transfers resume
Frequently appended table	Every append rehashes the whole thing	Partition by date; track partitions separately



Data that lives in a
warehouse

DVC would duplicate it

Track the query and its result hash,
not the bytes

The sharding pattern, which is the single most effective change for image and audio datasets:

dvc.yaml

Copy

```
stages:
  shard:
    cmd: python src/shard.py --in data/raw-images --out data/shards --size 512
    deps: [src/shard.py, data/raw-images]
    outs:
      - data/shards          # ~2,000 tar files instead of 1,000,000 jpegs
```

That gives DVC two thousand objects instead of a million: `dvc add` drops from tens of minutes to under one, the `.dir` listing shrinks by three orders of magnitude, and transfers become throughput-bound rather than request-bound. Your data loader gets faster too, because sequential reads beat a million opens.

Other levers at this scale:

BASH

Copy

```
dvc remote modify origin jobs 32          # transfer parallelism
dvc config core.checksum_jobs 16           # hashing parallelism
dvc remote modify origin version_aware true # natural paths + bucket versioning
dvc config cache.type reflink,hardlink     # avoid duplicating TBs locally
dvc add --to-remote data/huge.parquet     # transfer straight to the remote
dvc import-url --no-download s3://.../huge.parquet # track without pulling
```

`version_aware` matters at this scale: instead of hash-addressed objects, DVC stores files at their natural paths and relies on the bucket's own versioning for history. That makes the bucket browsable and readable by non-DVC tools, a real benefit when a data engineering team needs the same files, at the cost of losing cross-file deduplication.

There is a point where DVC is the wrong tool



travel over that shape natively, and DVC would be duplicating the warehouse. Track the **query and the result's hash** so your pipeline is reproducible, and let the lakehouse version the table. Saying this is a senior signal; insisting DVC handles everything is not.

► TRY IT

1. Time `dvc add` on 100,000 small files. Then shard them into 200 tars and time it again.
2. Compare the two `.dir` object sizes and the object counts pushed to the remote.
3. Raise `core.checksum_jobs` and `jobs` and re-measure both hashing and transfer.
4. Try `dvc add --to-remote` on a large file and confirm it never lands in the local cache.

Expected result – usually more than a tenfold improvement from sharding alone, and a `.dir` listing that shrinks from megabytes to kilobytes. Having those numbers for your own storage is what lets you make the sharding argument to a team.

Data registries as a product

Mid used `dvc import` to consume a dataset. Owning the producing side is a different job, and treating it as a product rather than a folder is what makes it adopted.

a registry repository

Copy

```
data-registry/
├── .dvc/config                # the canonical remote
├── datasets/
│   ├── customers/
│   │   ├── 2024-05.parquet.dvc
│   │   └── README.md        # schema, provenance, licence, owner
│   └── transactions/
│       └── 2024-05.parquet.dvc
├── CODEOWNERS
└── README.md                # how to consume, and the release policy
```



```
# Producer: publish a dated, immutable version and tag it
dvc add datasets/customers/2024-05.parquet
git add datasets/customers/2024-05.parquet.dvc
git commit -m "customers: 2024-05 snapshot, 4.2M rows"
git tag customers-2024.05
dvc push && git push --tags

# Consumer: pin an explicit revision
dvc import --rev customers-2024.05 \
  git@github.com:org/data-registry.git datasets/customers/2024-05.parquet

# Consumer: update deliberately, later
dvc update --rev customers-2024.06 datasets/customers/2024-05.parquet.dvc
```

What makes it a product rather than a bucket:

Property	Why it matters
Dated, immutable paths	<code>2024-05.parquet</code> , never <code>latest.parquet</code> . A consumer's pin stays meaningful
Git tags per release	<code>--rev</code> targets a human-readable version, not a SHA
A README per dataset	Schema, row count, provenance, licence, owner, known issues
desc: on outputs	<code>dvc list --dvc-only</code> becomes self-documenting
CODEOWNERS	Someone is accountable for each dataset
A deprecation policy	Consumers get notice before a version disappears
A validation stage	Schema and row-count checks run before publication

dvc.yaml in the registry

Copy

```
stages:
  validate:
    cmd: python validate.py datasets/customers/2024-05.parquet
    deps: [validate.py, datasets/customers/2024-05.parquet]
```



The failure mode of a shared registry

A team needs one extra column, cannot get it added quickly, and forks the dataset. Six months later there are nine variants and no canonical source. The fix is **turnaround time on requests**, not policy. If a reasonable schema change takes three weeks, forking is the rational choice and you will lose. Treat the registry as a product with a service level, or do not call it canonical.

► TRY IT

1. Create a small registry repository with one dated dataset, a README describing its schema, and a Git tag.
2. From a second repository, `dvc import --rev` that tag and confirm the pointer records it.
3. Publish a second version, tag it, and run `dvc update --rev` in the consumer.
4. Add a validation stage writing row counts to a `cache: false` metrics file, and check it diffs across the two versions.

Expected result – a consumer pinned to a named release, updating only when it chooses, with a machine-readable record of what changed between versions. That validation diff is what makes an upstream schema change visible instead of a Monday-morning surprise.

Lineage, provenance, and audit

`dvc.lock` already records which inputs produced which outputs. For regulated work you need to turn that into an answer a reviewer or an auditor accepts, and the gap between the two is smaller than it looks, but it is not zero.

The question is always some form of: "This model made a decision affecting a customer. Prove what trained it."



```
# Which commit is this model from? Start with its hash.
git log --all -S "9e8d7c6b5a4f3e2d" -- dvc.lock

# Then reconstruct that revision completely
git checkout <sha>
dvc pull
dvc repro --dry          # confirm nothing is out of date

# What differed between two revisions?
dvc diff <old-sha> <new-sha>
dvc params diff <old-sha> <new-sha>
dvc metrics diff <old-sha> <new-sha>
```

Question	Answered by
Which data trained this model?	<code>dvc.lock</code> at that commit: every dep hash
Which code version?	The Git commit itself
Which parameters?	<code>params.yaml</code> at that commit
What were the metrics?	<code>metrics.json</code> , committed in Git
Who approved it?	The pull request and its reviewers
Can we rebuild it byte-for-byte?	Only if the environment was pinned too

That last row is the honest gap. `dvc.lock` pins your data and your code; it does **not** pin your Python version, your CUDA driver, or the exact wheel of scikit-learn that was resolved.

Closing it takes three things:

`dvc.yaml`

Copy

```
stages:
  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - requirements.lock          # fully pinned, hash-verified
```



Pin the dependency set with hashes (`pip-compile --generate-hashes` , `poetry.lock` , `uv.lock`). **Declare it as a dependency**, so a change invalidates the stage. **Record the container digest**, not a tag, so the environment is addressable:

src/train.py

Copy

```
live.log_params({
    "git_sha": subprocess.check_output(["git", "rev-parse", "HEAD"]).decode().strip(),
    "image_digest": os.environ.get("IMAGE_DIGEST", "unknown"),
    "data_hash": read_lock_hash("data/train.parquet"),
})
```

The strongest audit answer is a rebuild

Documentation of lineage is weaker than a demonstration of it. A scheduled job that checks out a random past release, runs `dvc pull` and `dvc repro` , and asserts the metrics match is worth more than any lineage diagram, because it fails loudly the day reproducibility breaks, rather than the day an auditor asks.

► TRY IT

1. Pick a model artifact from three months ago and, using only its hash, find the commit that produced it.
2. Check that revision out, `dvc pull` , and run `dvc repro --dry` . Did it report up to date?
3. List everything that revision does

Expected result – not

pin: Python version, wheel versions, GPU driver, container digest.

4. Add a hash-pinned lock file and the Dockerfile to a stage's `deps` , and confirm changing either invalidates the stage.

Expected result – a model traced back to a commit in one command, and an honest list of what is still unpinned. Step three is the useful one: most teams discover their reproducibility stops at the environment boundary.



This is the hardest problem in the guide, and precision matters because the naive answer is wrong.

A subject exercises their right to erasure. Their records are in `customers-2024-03.parquet`, which is in your DVC cache, on your remote, in three developers' local caches, and referenced by every commit that trained a model on it.

Deleting the object breaks history. Every commit referencing that hash now fails to check out. **Not deleting it violates the law.** And rewriting Git history to remove the pointer does not touch the data at all.

✓ What works: design for erasure

- Keep personal data out of the versioned store entirely: version **derived, aggregated, or pseudonymised** data
- Crypto-shredding: encrypt per subject, delete the key
- Partition by subject or cohort so deletion is scoped
- Keep raw personal data in a governed system with real deletion, and let DVC track only its hash

✗ What does not work

- Deleting the object and hoping nothing referenced it
- Rewriting Git history, since the bytes are elsewhere
- Compliance-mode object lock plus an erasure obligation
- "We will handle it when someone asks"

If you are already in the bad situation, the sequence is:

01 Establish the blast radius

Which hashes contain the data, and which commits, tags, and models reference those hashes. This is a `git log -S` over `dvc.lock` and every `.dvc` file.

02 Decide what must survive

Usually the model and its metrics must remain auditable while the training data must go. That is an acceptable outcome in most regimes, and it needs to be written down.

03 Rebuild the dataset without the subject



04 Delete the old objects everywhere

Remote, all local caches, all CI caches, and any registry mirror. Old versions in a versioned bucket count.

05 Record the tombstone

A committed note stating which hash was erased, when, why, and under what authority. An unresolvable pointer with no explanation is worse than one with a documented reason.

06 Close the class

Move personal data out of the versioned store so the next request is a key deletion rather than an incident.

Content addressing and erasure are in tension

A content-addressed, immutable, widely replicated store is exactly the wrong shape for "make this specific record disappear everywhere". Git has the same tension, and so do backups and any append-only log. The resolution is always architectural and always upstream of the versioning tool: **do not put erasable data in an immutable store.**

► TRY IT

1. Take one of your datasets and classify it: does it contain data a subject could demand be erased?
2. If yes, trace how many commits and models reference it. Write the number down.
3. Design the alternative: what would the pipeline look like if the versioned artifact were pseudonymised or aggregated instead?
4. Write the one-page erasure runbook for your own project, including who authorises it.

Expected result – usually an uncomfortable number in step two, and a redesign in step three that is smaller than expected. Writing the runbook before you need it is the whole exercise. This is not a problem to think about for the first time under a thirty-day deadline.



Mid put `dvc repro` in CI on a hosted runner. Real training needs GPUs, long timeouts, and data locality, and reporting that a reviewer reads.

.github/workflows/train.yml

Copy

```
name: Train
on:
  pull_request:
  workflow_dispatch:

permissions:
  id-token: write
  contents: read
  pull-requests: write

jobs:
  train:
    runs-on: [self-hosted, gpu]          # or a cloud runner provisioned per job
    timeout-minutes: 480
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ vars.DVC_ROLE_ARN }
          aws-region: ${ vars.AWS_REGION }

      - run: pip install -r requirements.lock --require-hashes
      - run: dvc pull --allow-missing -j 32
      - run: dvc repro

      - name: Comment the report on the pull request
        env:
          REPO_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: |
          {
            echo "## Metrics"
            dvc metrics diff origin/main --md
            echo "## Parameters"
            dvc params diff origin/main --md
```



```
dvc plots diff origin/main --show-vega > plots.json || true
cml comment create report.md
```

```
- run: dvc push
  if: github.ref == 'refs/heads/main'
```

Four scale concerns and their answers:

Concern	Answer
Training needs a GPU for six hours	Self-hosted runner, or a cloud runner provisioned per job and destroyed after
Data locality	Runner in the same region as the bucket: egress often exceeds compute cost
Cold start on every run	Persistent runner with a warm <code>.dvc/cache</code> , or a cache action keyed on <code>dvc.lock</code>
Reviewer visibility	CML comments with metrics, parameter, and data diffs plus rendered plots

Self-hosted runners keep state, and that is the risk

A persistent runner with a warm cache is fast and also a shared, writable environment that has run every contributor's code. Make them **ephemeral** for anything triggered by a pull request, keep the warm-cache runners on protected branches only, and never let a fork-triggered job hold the data remote's write credential.

► TRY IT

1. Add CML reporting to an existing pipeline workflow and open a pull request that changes a parameter.
2. Confirm the comment includes metrics, parameters, and a data diff.
3. Measure the run with a warm `.dvc/cache` against a cold one.
4. Check what identity your pull-request job holds, and whether a fork build would receive it.



MOST TEAMS HAVE NEVER VERIFIED.

Where DVC stops

Naming the boundary is itself a senior signal, and it lands better than defending DVC for every shape of data.

✓ DVC is the right tool

- File-shaped data: images, audio, text, model artifacts
- Data versioned alongside the code that reads it
- Pipelines where reruns should be incremental
- Experiment tracking tied to Git history
- Teams that already live in pull requests

✗ Reach for something else

- Billion-row tables queried with SQL → Iceberg, Delta, Hudi
- Low-latency online features → a feature store
- Streaming and continuous ingestion → the streaming platform's own semantics
- Fine-grained row-level access control → a governed warehouse
- Hundreds of concurrent writers → a database

Tool	Solves	Overlaps DVC on
Iceberg / Delta / Hudi	Time travel and schema evolution over huge tables	Data versioning, but for tables, not files
A feature store	Consistent features online and offline, point-in-time correctness	Nothing much; it is a serving concern
MLflow / W&B	Experiment tracking and a model registry, with a UI	Experiments and metrics; DVC ties them to Git instead
LakeFS	Git-like branching over an entire object store	Data versioning, at bucket rather than repo granularity



Airflow / Dagster /
Prefect

Scheduling, retries, distributed
orchestration

Pipelines, but DVC has no
scheduler

The combination that works in practice: **DVC for file-shaped data and reproducible pipelines, a table format for warehouse-shaped data, an orchestrator for scheduling, and DVC's own experiment tracking or MLflow for the model registry.** They compose because they solve different problems; the mistake is expecting any one of them to do all four.

The honest answer about the DVC-plus-orchestrator question

DVC has no scheduler, no retries, and no distributed execution. For a nightly retrain you run `dvc repro` from Airflow or GitHub Actions: the orchestrator schedules, DVC decides what needs to run and guarantees the lineage. Presenting DVC as an Airflow replacement is a red flag in an interview; presenting it as the reproducibility layer underneath one is correct.

► TRY IT

1. List your project's datasets and classify each as file-shaped or table-shaped.
2. For any table-shaped one, work out what you would lose by moving it to a table format and tracking only the query.
3. Write down the specific requirement that would make you add an orchestrator, and what DVC would still be responsible for.
4. Name the one thing in your stack that currently has no versioning story at all.

Expected result — a clear split between what belongs in DVC and what does not, and usually one unversioned component nobody had noticed. That last answer is often the most valuable thing on this page.

Running DVC as a platform



Publish a template repository, not documentation. A `dvc.yaml` skeleton, a `.dvc/config` with the right remote and cache settings, a CI workflow with OIDC and the `dvc status` gate, and a `params.yaml`. Teams inherit every good decision by cloning; a wiki page is read once and ignored.

Own the remote layout and its policy. One prefix per environment, one bucket policy, versioning on, delete removed from human credentials, lifecycle rules for retention. Teams should not each invent this.

Make the safe path the fast path. If `dvc pull` from the sanctioned remote is fast and well-documented while copying from a colleague's laptop is slow, nobody copies from laptops. Regional co-location and a tuned `jobs` setting do more for compliance than a policy statement.

Measure and publish the numbers. Without them, "our data is under control" is an opinion:

Metric	Why it matters
Total bytes and objects per project	Cost, and the growth trend
Fraction of bytes in unreferenced experiments	The easiest saving available
Percentage of pipelines where <code>dvc status</code> is clean in CI	Whether reproducibility is real
Age of the oldest successfully reproduced release	The genuine reproducibility horizon
Number of remotes with human write access	The blast radius
Median <code>dvc pull</code> time in CI	Developer and CI friction

The scheduled reproduction check is the metric that matters

Pick a random past release each week, check it out on a clean runner, `dvc pull`, `dvc repro`, and assert the metrics match. That single job converts "reproducible" from a claim into a monitored property, and it fails on the day an upstream dependency, a deleted object, or an unpinned library breaks it, rather than on the day someone needs it.



1. Build a template repository containing a working pipeline, the CI workflow, and the shared `.dvc/config`. Use it for one real project.
2. Collect two of the metrics above across your repositories. Unreferenced experiment bytes is usually the most revealing.
3. Add the scheduled reproduction check for one release and let it run for a fortnight.
4. Write the one-paragraph policy you would publish alongside the template, including how fast you will answer a request to change it.

Expected result – a template that makes the right thing the default, and at least one metric with a number nobody expected. Step four is what determines whether the template is adopted or forked.

Incident playbooks

Under pressure, ordering matters more than knowledge. Five shapes, with the first move for each.

A remote object is corrupt or missing

Confirm it first: `dvc pull -v` names the hash and the remote. Then check whether bucket versioning has a previous version of that key; if so, restore it and you are done. If not, find any machine or CI cache that still holds the object (`find .dvc/cache -name '<rest-of-hash>'`) and push from there. If nothing has it, the data is gone: identify every commit referencing the hash, rebuild the artifact if its inputs survive, and record a tombstone if it cannot be rebuilt. Then turn on `verify true` and versioning so the next occurrence is recoverable.

`dvc gc --cloud` deleted live data

Stop all pushes immediately, because a subsequent push can overwrite the delete markers you need. Restore from bucket versioning. This is the entire reason versioning is non-optional.



A model cannot be reproduced

Work through the layers in order, because each is cheaper to check than the next: is the code commit findable, does `dvc pull` succeed for every dep in `dvc.lock`, does `dvc repro --dry` report up to date, and does an actual rerun produce matching metrics. The break is almost always the last two, and almost always the environment: an unpinned wheel, a different CUDA version, an undeclared random seed. Fix it by adding the missing pin to `deps`, not by documenting the discrepancy.

The cache filled the disk mid-sweep

- 0m Stop the queue**
`dvc queue stop`. Do not delete anything yet, because a half-written cache object is worse than a full disk.
- 2m Measure before deleting**
`du -sh .dvc/cache` and `dvc gc --dry-run --all-commits`. Know what is reachable.
- 5m Drop experiment refs first**
`dvc exp remove --all` makes sweep artifacts unreachable, then `dvc gc --all-commits` reclaims them safely.
- 10m Move the cache if needed**
`dvc cache dir --local /mnt/big/cache` rather than fighting a small root disk.
- after Close the loop**
Mark regenerable intermediates `push: false`, budget disk per sweep, and schedule experiment cleanup.

A credential leaked

Rotate before anything else, because the credential may already be in a log, a cache, or a fork. Then audit the bucket's access logs for the exposure window, looking specifically for writes and deletes rather than only reads. Replace the mechanism, not just the value: if a



► TRY IT

1. On a test bucket, delete an object DVC needs, run `dvc pull`, and restore it from bucket versioning.
2. Run `dvc gc --cloud --dry-run --workspace` on a repository with several branches and read how much it would destroy.
3. Fill a small disk with a sweep on purpose, then recover using the timeline above.
4. Write your own version of these playbooks, adapted to your storage provider, and put them where an on-call person will find them.

Expected result – a recovered object, an alarming dry-run number, and a runbook that exists before you need it. Step two is the one to show anyone who has delete permission on your remote.

The review checklist

Take this checklist away from this level. Run it against any DVC project (your own, or one you are reviewing) and it catches nearly everything covered here before it becomes an incident.

Check	Looking for	Level
Is <code>dvc.lock</code> committed and never hand-edited?	The record of what ran	Beginner
Are all scripts and config files in <code>deps</code> ?	No silent skips	Beginner
Is <code>params.yaml</code> used instead of hard-coded values?	Config changes invalidate stages	Beginner
Do metrics and plots use <code>cache: false</code> ?	Diffable in Git without a pull	Beginner
Is <code>.dvc/config.local</code> gitignored and credential-free elsewhere?	No secret in history	Beginner



Are regenerable intermediates marked <code>push: false</code> ?	Storage cost bounded	Mid
Is the environment pinned and declared as a dependency?	Reproducibility beyond code and data	Senior
Are external deps immutable, dated paths?	A moving source cannot rewrite history	Mid
Is <code>verify true</code> set on remotes?	Corruption is caught, not restored silently	Senior
Is bucket versioning enabled?	A mistaken delete is survivable	Senior
Can any human credential delete remote objects?	The <code>dvc gc --cloud</code> blast radius	Senior
Does CI use OIDC rather than a stored key?	Nothing to leak	Senior
Do fork pull requests get read-only access?	A contributor cannot overwrite data	Senior
Is there a <code>dvc status</code> gate in CI?	The lock cannot drift from the code	Mid
Is retention a lifecycle policy, not a manual <code>gc</code> ?	Audited, and not triggerable by a stale clone	Senior
Are small-file datasets sharded?	Hashing and request costs bounded	Mid
Does any versioned artifact contain erasable personal data?	The erasure problem, before it is urgent	Senior
Is there a scheduled reproduction check?	Reproducibility is monitored, not assumed	Senior
Is each dataset's owner recorded?	Someone is accountable	Senior
Is the storage cost visible to the team?	Growth gets managed	Senior



in `.dvc/config`, a check that `dvc.lock` is committed whenever `dvc.yaml` changes, and a script asserting bucket versioning is on. Put those in CI and reserve human review for the judgement calls: is that external dep immutable, does this dataset contain personal data, is ``push: false`` the right call here.

The complete picture

The series' final artefact: every level's topics, hardened. Nothing in it is new.

dvc.yaml

Copy

```
vars:
  - params.yaml

stages:
  # Immutable dated source: reproducible in eighteen months
  ingest:
    cmd: python src/ingest.py
    deps:
      - src/ingest.py
      - requirements.lock
      - s3://landing/exports/2024-05-01/events.parquet
    outs:
      - data/raw.parquet

  # Sharded once, so hashing and transfer stay bounded
  shard:
    cmd: python src/shard.py --size 512
    deps: [src/shard.py, data/raw.parquet]
    outs:
      - data/shards

  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py, requirements.lock, data/shards]
```



```
- data/test.parquet:
    remote: archive          # cold storage: read rarely, kept for audit

featurize:
  foreach: ${featurize.methods}
  do:
    cmd: python src/featurize.py --method ${item}
    deps: [src/featurize.py, data/train.parquet]
    params: [featurize.max_features]
    outs:
      - data/features-${item}.pkl:
          push: false          # four minutes to rebuild; do not store 200 GB

train:
  cmd: python src/train.py
  deps:
    - src/train.py
    - requirements.lock        # hash-pinned; a wheel change invalidates the stage
    - Dockerfile               # the environment, as code
    - data/features-tfidf.pkl
  params: [train.n_estimators, train.max_depth, train.learning_rate]
  outs:
    - models/model.pkl
    - logs/train.log:
        persist: true
  metrics:
    - dvclive/metrics.json:
        cache: false
  plots:
    - dvclive/plots/metrics:
        x: step
        cache: false

evaluate:
  cmd: python src/evaluate.py
  deps: [src/evaluate.py, models/model.pkl, data/test.parquet]
  metrics:
    - metrics.json:
        cache: false
  plots:
    - plots/confusion.csv:
        template: confusion
        x: actual
```



BASH

Copy

```
# .dvc/config - committed. No credentials anywhere in it.
[core]
    remote = origin
    autostage = true
    checksum_jobs = 16
['remote "origin"']
    url = s3://ml-data-prod/dvc
    jobs = 32
    verify = true                # corruption is caught, not restored
    sse = AES256
['remote "archive"']
    url = s3://ml-data-archive/dvc
    verify = true
[cache]
    type = reflink,copy
    protected = true            # in-place edits fail loudly
```

BASH

Copy

```
# Verify, do not assume
dvc status && dvc status -c          # local and remote agreement
dvc pull --verify                    # re-hash everything on download
dvc repro --dry                      # nothing stale
git log --all -S "$MODEL_HASH" -- dvc.lock # trace a model to its commit
dvc gc --cloud --dry-run --all-commits    # never without --dry-run first
aws s3api get-bucket-versioning --bucket ml-data-prod # confirm survivability
```

► TRY IT: THE FINAL EXERCISE

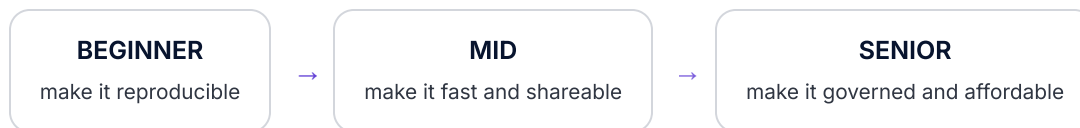
1. Build this end to end on a real project: pinned environment declared as a dependency, sharded inputs, tiered remotes, OIDC in CI, and the `dvc status` gate.
2. Verify each control actively rather than trusting it: corrupt an object and confirm `verify` refuses it, try to delete from the remote with a developer credential and confirm refusal, try to edit a protected output in place, and push from a fork build and confirm it fails.
3. Reproduce a release from three months ago on a clean runner and confirm the metrics match.



control you have never watched refuse anything is decoration. Step two is the difference between a project that looks governed and one that is. The colleague review will find something you did not, which is the point.

Where the series leaves you

Across the three levels you have gone from `dvc add` on a CSV to owning data versioning as a platform. The same topics carried all the way through, each time with more depth:



You should now be able to look at a DVC project and see not just what it does but what it **guarantees**: whether a result from last year can be rebuilt, whether the environment is pinned as well as the data, who can overwrite the bytes a model was trained on, whether a corrupted object would be noticed, whether a mistaken `gc` is survivable, what it costs per month, and whether the versioned artifacts contain data somebody could lawfully demand you erase.

Can you...	
Explain what a <code>.dvc</code> file contains and why?	A hash, size, path: Git-sized, content-addressed
Say how <code>dvc repro</code> decides what to run?	Hashes in <code>dvc.lock</code> versus current inputs
Name the most dangerous DVC command?	<code>dvc gc --cloud --workspace</code>
Say why write access to a remote is a big deal?	An object at a hash path can be replaced silently
Give the mechanism that catches corruption?	<code>verify true</code> , and bucket versioning to recover
Explain how CI gets credentials with nothing stored?	OIDC with a branch-pinned trust policy



Say what <code>dvc.lock</code> does <code>not</code> pin:	The environment. pin and declare it
Name the fix for a million small files?	Shard them; measure the difference
Describe a data registry as a product?	Dated immutable paths, tags, README, owner, SLA
Explain why erasure conflicts with content addressing?	Immutable, replicated, hash-addressed by design
Say where DVC stops?	Tables, streaming, online features, scheduling
Name the metric that proves reproducibility?	A scheduled rebuild of a past release

That review instinct outlasts any specific command, because, as with pipelines and containers, most data incidents are prevented at review time rather than at recovery time.



PREVIOUS

Mid-level · Tips & Tricks

NEXT UP

Senior · Interview Review





LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

